MEDIGUARD PROJECT

Complete Implementation Guide

Secure Federated Health Intelligence Platform

A Privacy-Preserving Machine Learning System for Healthcare

Contents

Chapter 1

Executive Summary

1.1 Project Objective

Build a privacy-preserving federated machine learning system for healthcare that complies with HIPAA/GDPR regulations, using only free and open-source software.

1.2 Problem Statement

Hospitals globally generate valuable patient data that could power AI-driven diagnostics, outbreak prediction, and personalized medicine. However, legal constraints (HIPAA, GDPR) and cybersecurity threats prevent hospitals from sharing raw patient data, even with other hospitals or research institutions.

1.3 Solution: MediGuard

MediGuard enables multiple hospitals to collaboratively train machine learning models **WITHOUT** sharing raw patient data. Each hospital keeps its data local and encrypted, sharing only model weights through a secure federated learning architecture.

1.4 Key Innovations

- **Federated Learning:** Train AI across hospitals without data leaving their networks
- **Zero-Knowledge Proofs:** Prove data properties without revealing actual values
- **Custom PKI:** Certificate-based authentication for all communications
- **AES-256 Encryption:** All sensitive data encrypted at rest
- **Network Segmentation:** VPN + firewall rules enforce strict access control
- **Real-time Monitoring:** Dashboard displays security events and ML progress

1.5 Success Metrics

Metric	Target	Purpose
VPN Latency	<10ms	Network performance
DB Query Speed	<100ms	Database efficiency
Encryption Coverage	100% PII fields	Security compliance
FL Model Accuracy	>85%	ML effectiveness
Audit Log Coverage	100% queries	Compliance requirement
Zero-Knowledge Proof Success	100% validations	Privacy guarantee

Table 1.1: Success Metrics and Targets

1.6 Technology Stack Summary

All technologies used are free, open-source, and production-grade:

- **OS:** Ubuntu 24.04 LTS
- **Virtualization:** VirtualBox
- **VPN:** WireGuard
- **Firewall:** UFW
- **Database:** PostgreSQL 16
- **Encryption:** OpenSSL, Python cryptography
- **ML Framework:** PyTorch, scikit-learn, XGBoost
- **Federated Learning:** Flower (flwr)
- **NLP:** HuggingFace Transformers + BioBERT
- **Backend:** FastAPI
- **Frontend:** React + Tailwind CSS

Chapter 2

Project Overview

2.1 Architecture Diagram

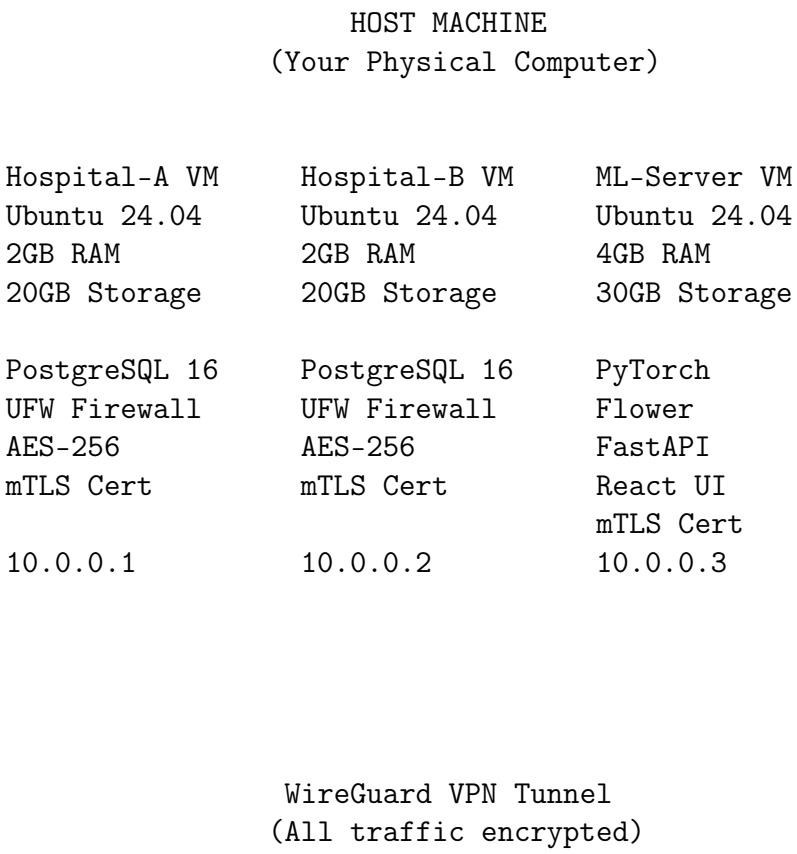

Figure 2.1: System Architecture Overview

2.2 Data Flow Example

Scenario: Doctor queries patient readmission risk

1. Browser → Dashboard (React UI)
2. Dashboard → FastAPI Backend (REST API call)
3. FastAPI → Verifies mTLS certificate
4. FastAPI → Queries Hospital-A DB via VPN (encrypted tunnel)
5. PostgreSQL → Returns AES-encrypted patient data
6. FastAPI → Decrypts data in memory (never written to disk)
7. FastAPI → Feeds data to ML model
8. ML Model → Predicts readmission risk (0.82 = 82% probability)
9. FastAPI → Encrypts result + signs with RSA key
10. Dashboard → Displays risk score to doctor

2.2.1 Security Layers Applied

✓ VPN encryption (WireGuard)

✓ mTLS authentication (PKI certificates)

✓ Database encryption (AES-256-GCM)

✓ Digital signature (RSA-PSS)

✓ Firewall rules (UFW)

✓ Audit logging (append-only)

2.3 Hardware Requirements

Resource	Minimum	Recommended	Notes
RAM	8 GB	16 GB	3 VMs running simultaneously
Storage	100 GB	150 GB	Databases + ML models
CPU	4 cores	6-8 cores	Parallel ML training
Host OS	Any	Linux	Windows/Mac/Linux all supported

Table 2.1: Hardware Requirements

Chapter 3

PHASE 1: Infrastructure Setup

Phase Details

Duration: Week 1
Goal: Build encrypted network foundation with 3 VMs connected via VPN and protected by firewalls

3.1 VirtualBox Installation

3.1.1 On Ubuntu/Debian Host

```
sudo apt update
sudo apt install virtualbox virtualbox-ext-pack -y
```

3.1.2 On Windows Host

1. Download VirtualBox from https://www.virtualbox.org/
2. Install with default settings
3. Download Extension Pack and install via File → Preferences → Extensions

3.1.3 On macOS Host

```
brew install --cask virtualbox
brew install --cask virtualbox-extension-pack
```

3.2 Ubuntu Server VM Setup

3.2.1 Download Ubuntu 24.04 LTS Server ISO

```
wget https://releases.ubuntu.com/24.04/ubuntu-24.04-live-server-amd64.iso
```

3.2.2 Create VM #1: hospital-a

VirtualBox GUI Steps:

1. Click "New"
2. Name: `hospital-a`
3. Type: Linux
4. Version: Ubuntu (64-bit)
5. Memory: 2048 MB
6. Create virtual hard disk: VDI, Dynamically allocated, 20 GB
7. Settings → Network → Adapter 1: NAT
8. Settings → Network → Adapter 2: Host-only Adapter (vboxnet0)
9. Settings → Storage → Add optical drive → Select Ubuntu ISO
10. Start VM → Follow Ubuntu installation wizard

Installation Choices:

- Profile name: `mediguard`
- Server name: `hospital-a`
- Username: `admin`
- Install OpenSSH server: **YES**
- No additional packages

3.2.3 Repeat for VM #2: hospital-b

Same steps, change name to `hospital-b`

3.2.4 Create VM #3: ml-server

Same steps but:

- Memory: 4096 MB (4 GB)
- Storage: 30 GB
- Name: `ml-server`

3.3 Post-Installation Setup

On all 3 VMs, run:

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install essential tools
sudo apt install -y \
    net-tools \
    curl \
    wget \
    vim \
    htop \
    git \
    build-essential \
    python3 \
    python3-pip \
    python3-venv

# Check network interfaces
ip addr show
```

Expected Output

You should see two network interfaces (e.g., `enp0s3` for NAT, `enp0s8` for host-only)

3.4 WireGuard VPN Setup

3.4.1 Install WireGuard on all 3 VMs

```
sudo apt install wireguard wireguard-tools -y
```

3.4.2 Generate Keys on each VM

On hospital-a:

```
cd /etc/wireguard
sudo umask 077
sudo wg genkey | sudo tee privatekey | wg pubkey | sudo tee
   publickey
```

Note the keys:

```
# Private key (keep secret!)
sudo cat privatekey

# Public key (share with other VMs)
```

```
sudo cat publickey
```

Repeat on `hospital-b` and `ml-server`

3.4.3 Configure WireGuard

On hospital-a, create `/etc/wireguard/wg0.conf`:

```
sudo vim /etc/wireguard/wg0.conf
```

```
[Interface]
Address = 10.0.0.1/24
PrivateKey = <hospital-a-private-key>
ListenPort = 51820

# Peer: hospital-b
[Peer]
PublicKey = <hospital-b-public-key>
AllowedIPs = 10.0.0.2/32
Endpoint = <hospital-b-host-only-IP>:51820

# Peer: ml-server
[Peer]
PublicKey = <ml-server-public-key>
AllowedIPs = 10.0.0.3/32
Endpoint = <ml-server-host-only-IP>:51820
```

On hospital-b, create `/etc/wireguard/wg0.conf`:

```
[Interface]
Address = 10.0.0.2/24
PrivateKey = <hospital-b-private-key>
ListenPort = 51820

# Peer: hospital-a
[Peer]
PublicKey = <hospital-a-public-key>
AllowedIPs = 10.0.0.1/32
Endpoint = <hospital-a-host-only-IP>:51820

# Peer: ml-server
[Peer]
PublicKey = <ml-server-public-key>
AllowedIPs = 10.0.0.3/32
Endpoint = <ml-server-host-only-IP>:51820
```

On ml-server, create `/etc/wireguard/wg0.conf`:

```
[Interface]
Address = 10.0.0.3/24
```

```
PrivateKey = <ml-server-private-key>
ListenPort = 51820

# Peer: hospital-a
[Peer]
PublicKey = <hospital-a-public-key>
AllowedIPs = 10.0.0.1/32
Endpoint = <hospital-a-host-only-IP>:51820

# Peer: hospital-b
[Peer]
PublicKey = <hospital-b-public-key>
AllowedIPs = 10.0.0.2/32
Endpoint = <hospital-b-host-only-IP>:51820
```

Replace placeholders

- `<X-private-key>` → Output from `sudo cat /etc/wireguard/privatekey` on that VM
- `<X-public-key>` → Output from `sudo cat /etc/wireguard/publickey` on that VM
- `<X-host-only-IP>` → IP from `ip addr show enp0s8` (e.g., 192.168.56.101)

3.4.4 Start WireGuard on all VMs

```
sudo systemctl enable wg-quick@wg0
sudo systemctl start wg-quick@wg0

# Verify interface is up
sudo wg show
```

Expected output:

```
interface: wg0
  public key: <your-public-key>
  private key: (hidden)
  listening port: 51820

peer: <peer-public-key>
  endpoint: 192.168.56.102:51820
  allowed ips: 10.0.0.2/32
```

3.5 VPN Testing

From hospital-a, ping hospital-b:

```
ping -c 4 10.0.0.2
```

Expected output:

```
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.521 ms
```

From ml-server, ping both hospitals:

```
ping -c 4 10.0.0.1
ping -c 4 10.0.0.2
```

Success Criteria

All pings succeed with <10ms latency

3.6 UFW Firewall Setup

3.6.1 On hospital-a

```
# Reset firewall to default
sudo ufw --force reset

# Default policies: deny all incoming, allow outgoing
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Allow SSH (so you don't lock yourself out!)
sudo ufw allow 22/tcp

# Allow WireGuard
sudo ufw allow 51820/udp

# Allow PostgreSQL ONLY from hospital-b and ml-server
sudo ufw allow from 10.0.0.2 to any port 5432 proto tcp
sudo ufw allow from 10.0.0.3 to any port 5432 proto tcp

# Deny PostgreSQL from all other IPs
sudo ufw deny 5432/tcp

# Enable firewall
sudo ufw enable

# Verify rules
sudo ufw status numbered
```

Expected output:

```
Status: active

     To                         Action      From
     --                         ------      ----
```

```
[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 51820/udp                  ALLOW IN    Anywhere
[ 3] 5432/tcp                   ALLOW IN    10.0.0.2
[ 4] 5432/tcp                   ALLOW IN    10.0.0.3
[ 5] 5432/tcp                   DENY IN     Anywhere
```

3.6.2 On hospital-b

```
sudo ufw --force reset
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 51820/udp
sudo ufw allow from 10.0.0.1 to any port 5432 proto tcp
sudo ufw allow from 10.0.0.3 to any port 5432 proto tcp
sudo ufw deny 5432/tcp
sudo ufw enable
sudo ufw status numbered
```

3.6.3 On ml-server

```
sudo ufw --force reset
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 51820/udp
sudo ufw allow 3000/tcp  # React dashboard
sudo ufw allow 8000/tcp  # FastAPI backend
sudo ufw enable
sudo ufw status numbered
```

3.7 Firewall Testing

From ml-server, test PostgreSQL access to hospital-a:

```
# This should work (allowed IP)
nc -zv 10.0.0.1 5432
```

Expected: Connection succeeded!

From your host machine, test PostgreSQL access:

```
# This should FAIL (not allowed IP)
nc -zv 10.0.0.1 5432
```

Expected: Connection refused or timeout

Success Criteria

Firewall allows only whitelisted IPs

3.8 SSH Key Setup

On your host machine:

```
# Generate SSH key (if you don't have one)
ssh-keygen -t ed25519 -C "mediguard-admin"

# Copy key to all VMs
ssh-copy-id admin@<hospital-a-host-only-IP>
ssh-copy-id admin@<hospital-b-host-only-IP>
ssh-copy-id admin@<ml-server-host-only-IP>

# Test passwordless login
ssh admin@<hospital-a-host-only-IP>
```

3.9 Phase 1 Verification Checklist

- ☐ All 3 VMs boot successfully
- ☐ Each VM has 2 network interfaces (NAT + host-only)
- ☐ WireGuard VPN is running on all VMs (`sudo wg show`)
- ☐ All VMs can ping each other via VPN IPs (10.0.0.x)
- ☐ UFW firewall is active with correct rules
- ☐ Firewall blocks unauthorized PostgreSQL access
- ☐ SSH key authentication works

3.10 Troubleshooting

Problem	Solution
VMs can't ping each other	Check WireGuard config, verify public keys match
Firewall blocks legitimate traffic	Check rule order with `sudo ufw status numbered`
SSH connection refused	Verify OpenSSH server is running: `sudo systemctl status ssh`
VPN high latency (>50ms)	Check host machine CPU usage, close unnecessary applications

Table 3.1: Common Issues and Solutions

Chapter 4

PHASE 2: Database Layer

Phase Details

Duration: Weeks 2-3
Goal: Production-grade PostgreSQL databases with sharding, replication, stored procedures, and audit logging

4.1 PostgreSQL Installation

On `hospital-a` and `hospital-b` VMs:

```
# Install PostgreSQL 16
sudo apt install -y postgresql-16 postgresql-contrib-16

# Start and enable PostgreSQL
sudo systemctl enable postgresql
sudo systemctl start postgresql

# Verify installation
sudo -u postgres psql --version
```

Expected output: `psql (PostgreSQL) 16.x`

4.2 PostgreSQL Configuration

4.2.1 Configure PostgreSQL to listen on VPN interface

On hospital-a:

```
sudo vim /etc/postgresql/16/main/postgresql.conf
```

Find and modify:

```
listen_addresses = 'localhost,10.0.0.1'
max_connections = 100
shared_buffers = 256MB
```

4.2.2 Configure authentication

```
sudo vim /etc/postgresql/16/main/pg_hba.conf
```

Add these lines **BEFORE** the existing rules:

```
# Allow connections from VPN network
host    all             all             10.0.0.0/24
                scram-sha-256
```

4.2.3 Restart PostgreSQL

```
sudo systemctl restart postgresql
```

Important

Repeat on `hospital-b` (change `listen_addresses` to `'localhost,10.0.0.2'`)

4.3 Create Database and User

On hospital-a:

```
sudo -u postgres psql
```

```
-- Create database
CREATE DATABASE mediguard_hospital_a;

-- Create user with strong password
CREATE USER mediguard_admin WITH ENCRYPTED PASSWORD '
    YourSecurePassword123!';

-- Grant privileges
GRANT ALL PRIVILEGES ON DATABASE mediguard_hospital_a TO
    mediguard_admin;

-- Connect to database
\c mediguard_hospital_a

-- Grant schema privileges
GRANT ALL ON SCHEMA public TO mediguard_admin;

-- Exit
\q
```

Note

Repeat on `hospital-b` (change database name to `mediguard_hospital_b`)

4.4 Database Schema

Create file: `/home/admin/schema.sql`

```
-- ========================================
-- MediGuard Database Schema
-- Hospital Node Schema (v1.0)
-- ========================================

-- Enable extensions
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ========================================
-- TABLE: patients
-- Stores encrypted patient demographics
-- ========================================
CREATE TABLE patients (
    patient_id SERIAL PRIMARY KEY,
    uuid UUID DEFAULT uuid_generate_v4() UNIQUE NOT NULL,

    -- Encrypted PII fields (stored as bytea)
    name_encrypted BYTEA NOT NULL,
    ssn_encrypted BYTEA,
    dob_encrypted BYTEA NOT NULL,
    address_encrypted BYTEA,
    phone_encrypted BYTEA,

    -- Non-sensitive fields (plaintext)
    gender VARCHAR(10),
    blood_type VARCHAR(5),
    hospital_id VARCHAR(50) NOT NULL,

    -- Metadata
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    -- Indexes for performance
    CONSTRAINT chk_gender CHECK (gender IN ('M', 'F', 'Other', '
       Unknown'))
);

CREATE INDEX idx_patients_hospital ON patients(hospital_id);
CREATE INDEX idx_patients_created ON patients(created_at);

-- ========================================
-- TABLE: diagnoses
-- Stores patient diagnoses (ICD-10 codes)
-- ========================================
CREATE TABLE diagnoses (
```

```
    diag_id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL REFERENCES patients(patient_id) ON
        DELETE CASCADE,

    -- ICD-10 diagnosis code
    icd10_code VARCHAR(10) NOT NULL,
    description TEXT,

    -- Diagnosis details
    severity VARCHAR(20),
    diagnosed_date DATE NOT NULL,
    doctor_id INTEGER,

    -- Metadata
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_severity CHECK (severity IN ('Mild', 'Moderate',
        'Severe', 'Critical'))
);

CREATE INDEX idx_diagnoses_patient ON diagnoses(patient_id);
CREATE INDEX idx_diagnoses_icd10 ON diagnoses(icd10_code);
CREATE INDEX idx_diagnoses_date ON diagnoses(diagnosed_date);

-- ========================================
-- TABLE: prescriptions
-- Stores medication prescriptions
-- ========================================
CREATE TABLE prescriptions (
    rx_id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL REFERENCES patients(patient_id) ON
        DELETE CASCADE,

    -- Drug information
    drug_code VARCHAR(20) NOT NULL,  -- NDC or other standard code
    drug_name VARCHAR(200) NOT NULL,
    dosage VARCHAR(50) NOT NULL,
    frequency VARCHAR(50),
    duration_days INTEGER,

    -- Prescription details
    prescribed_date DATE NOT NULL,
    doctor_id INTEGER,
    pharmacy_id INTEGER,

    -- Metadata
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_prescriptions_patient ON prescriptions(patient_id);
CREATE INDEX idx_prescriptions_drug ON prescriptions(drug_code);
CREATE INDEX idx_prescriptions_date ON prescriptions(prescribed_date
    );
```

```
-- ========================================
-- TABLE: lab_results
```

```
-- Stores laboratory test results
-- ========================================
CREATE TABLE lab_results (
    lab_id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL REFERENCES patients(patient_id) ON
        DELETE CASCADE,

    -- Test information
    test_name VARCHAR(100) NOT NULL,
    test_code VARCHAR(20),
    result_value NUMERIC(10,2),
    result_unit VARCHAR(20),

    -- Reference ranges
    normal_range_min NUMERIC(10,2),
    normal_range_max NUMERIC(10,2),
    flag VARCHAR(20),  -- 'Normal', 'High', 'Low', 'Critical'

    -- Test details
    test_date DATE NOT NULL,
    lab_technician_id INTEGER,

    -- Metadata
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_flag CHECK (flag IN ('Normal', 'High', 'Low', '
        Critical'))
);

CREATE INDEX idx_lab_results_patient ON lab_results(patient_id);
CREATE INDEX idx_lab_results_test ON lab_results(test_name);
CREATE INDEX idx_lab_results_date ON lab_results(test_date);
CREATE INDEX idx_lab_results_flag ON lab_results(flag);

-- ========================================
-- TABLE: doctors
-- Stores doctor/provider information
-- ========================================
CREATE TABLE doctors (
    doctor_id SERIAL PRIMARY KEY,

    -- Doctor details
    name VARCHAR(200) NOT NULL,
    license_number VARCHAR(50) UNIQUE NOT NULL,
    specialty VARCHAR(100),
    department VARCHAR(100),
    hospital_id VARCHAR(50) NOT NULL,

    -- Contact
    email VARCHAR(200),
    phone VARCHAR(20),

    -- Metadata
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
CREATE INDEX idx_doctors_hospital ON doctors(hospital_id);
CREATE INDEX idx_doctors_specialty ON doctors(specialty);
```

```
-- ========================================
-- TABLE: audit_log
-- Append-only log of all database queries
-- ========================================
CREATE TABLE audit_log (
    log_id SERIAL PRIMARY KEY,

    -- Query information
    username VARCHAR(100) NOT NULL,
    query_text TEXT,
    query_hash VARCHAR(64) NOT NULL,  -- SHA-256 hash

    -- Connection details
    source_ip INET NOT NULL,
    source_port INTEGER,

    -- Timing
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    execution_time_ms INTEGER,

    -- Status
    status VARCHAR(20),  -- 'Success', 'Failed', 'Blocked'
    error_message TEXT
);

CREATE INDEX idx_audit_log_timestamp ON audit_log(timestamp);
CREATE INDEX idx_audit_log_username ON audit_log(username);
CREATE INDEX idx_audit_log_source_ip ON audit_log(source_ip);
CREATE INDEX idx_audit_log_status ON audit_log(status);

-- Prevent UPDATE and DELETE on audit_log (append-only)
CREATE RULE audit_log_no_update AS ON UPDATE TO audit_log DO INSTEAD
    NOTHING;
CREATE RULE audit_log_no_delete AS ON DELETE TO audit_log DO INSTEAD
    NOTHING;

-- ========================================
-- TABLE: network_alerts
-- Stores UFW firewall breach attempts
-- ========================================
CREATE TABLE network_alerts (
    alert_id SERIAL PRIMARY KEY,

    -- Attack details
    source_ip INET NOT NULL,
    dest_port INTEGER NOT NULL,
    protocol VARCHAR(10),

    -- Alert information
    alert_type VARCHAR(50),  -- 'Port Scan', 'Brute Force', 'SQL
       Injection'
    severity VARCHAR(20),
    blocked BOOLEAN DEFAULT TRUE,
```

```
51
52      -- Timing
53      timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
54
55      CONSTRAINT chk_severity_alert CHECK (severity IN ('Low', 'Medium
            ', 'High', 'Critical'))
56  );
57
58  CREATE INDEX idx_network_alerts_timestamp ON network_alerts(
        timestamp);
59  CREATE INDEX idx_network_alerts_source_ip ON network_alerts(
        source_ip);
60  CREATE INDEX idx_network_alerts_severity ON network_alerts(severity)
        ;
```

4.4.1 Stored Procedures

```
1   -- ========================================
2   -- STORED PROCEDURES
3   -- ========================================
4
5   -- Procedure 1: Flag high-risk patients
6   CREATE OR REPLACE FUNCTION flag_high_risk_patients()
7   RETURNS TABLE(patient_id INTEGER, risk_score NUMERIC, last_visit
        DATE) AS $$
8   BEGIN
9       RETURN QUERY
10      SELECT
11          p.patient_id,
12          (COUNT(DISTINCT d.diag_id) * 0.3 +
13           COUNT(DISTINCT lr.lab_id) * 0.2) AS risk_score,
14          MAX(d.diagnosed_date) AS last_visit
15      FROM patients p
16      LEFT JOIN diagnoses d ON p.patient_id = d.patient_id
17      LEFT JOIN lab_results lr ON p.patient_id = lr.patient_id
18      WHERE d.diagnosed_date < CURRENT_DATE - INTERVAL '90 days'
19      GROUP BY p.patient_id
20      HAVING COUNT(DISTINCT d.diag_id) > 3
21      ORDER BY risk_score DESC;
22  END;
23  $$ LANGUAGE plpgsql;
24
25  -- Procedure 2: Check drug interactions
26  CREATE OR REPLACE FUNCTION check_drug_interactions(p_patient_id
        INTEGER)
27  RETURNS TABLE(drug1 VARCHAR, drug2 VARCHAR, interaction_severity
        VARCHAR) AS $$
28  BEGIN
29      -- Simplified interaction checker
30      -- In production, this would query a comprehensive drug
            interaction database
31      RETURN QUERY
32      SELECT
33          p1.drug_name,
34          p2.drug_name,
```

```
        CASE
            WHEN p1.drug_code IN ('warfarin', 'aspirin') AND
                 p2.drug_code IN ('warfarin', 'aspirin')
            THEN 'Critical'
            ELSE 'Monitor'
        END AS interaction_severity
    FROM prescriptions p1
    JOIN prescriptions p2 ON p1.patient_id = p2.patient_id
    WHERE p1.patient_id = p_patient_id
      AND p1.rx_id < p2.rx_id
      AND p1.prescribed_date >= CURRENT_DATE - INTERVAL '30 days'
      AND p2.prescribed_date >= CURRENT_DATE - INTERVAL '30 days';
END;
$$ LANGUAGE plpgsql;
```

```
-- Procedure 3: Generate outbreak report
CREATE OR REPLACE FUNCTION generate_outbreak_report(
    p_region VARCHAR,
    p_month INTEGER
)
RETURNS TABLE(
    icd10_code VARCHAR,
    diagnosis_count BIGINT,
    avg_severity VARCHAR
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        d.icd10_code,
        COUNT(*) AS diagnosis_count,
        MODE() WITHIN GROUP (ORDER BY d.severity) AS avg_severity
    FROM diagnoses d
    JOIN patients p ON d.patient_id = p.patient_id
    WHERE p.hospital_id = p_region
      AND EXTRACT(MONTH FROM d.diagnosed_date) = p_month
      AND EXTRACT(YEAR FROM d.diagnosed_date) = EXTRACT(YEAR FROM
          CURRENT_DATE)
    GROUP BY d.icd10_code
    HAVING COUNT(*) > 10
    ORDER BY diagnosis_count DESC;
END;
$$ LANGUAGE plpgsql;

-- Procedure 4: Secure audit insert
CREATE OR REPLACE FUNCTION secure_audit_insert(
    p_username VARCHAR,
    p_query_text TEXT,
    p_source_ip INET
)
RETURNS VOID AS $$
DECLARE
    v_query_hash VARCHAR(64);
BEGIN
    -- Generate SHA-256 hash of query
    v_query_hash := encode(digest(p_query_text, 'sha256'), 'hex');

```

```
    -- Insert into append-only log
    INSERT INTO audit_log (username, query_text, query_hash,
        source_ip, status)
    VALUES (p_username, p_query_text, v_query_hash, p_source_ip, '
        Success');
END;
$$ LANGUAGE plpgsql;
```

4.4.2 Views for Analytics

```
-- ========================================
-- VIEWS FOR ANALYTICS
-- ========================================

-- View: Patient summary
CREATE OR REPLACE VIEW patient_summary AS
SELECT
    p.patient_id,
    p.uuid,
    p.gender,
    p.blood_type,
    p.hospital_id,
    COUNT(DISTINCT d.diag_id) AS total_diagnoses,
    COUNT(DISTINCT rx.rx_id) AS total_prescriptions,
    COUNT(DISTINCT lr.lab_id) AS total_lab_tests,
    MAX(d.diagnosed_date) AS last_diagnosis_date
FROM patients p
LEFT JOIN diagnoses d ON p.patient_id = d.patient_id
LEFT JOIN prescriptions rx ON p.patient_id = rx.patient_id
LEFT JOIN lab_results lr ON p.patient_id = lr.patient_id
GROUP BY p.patient_id, p.uuid, p.gender, p.blood_type, p.hospital_id
    ;

-- View: High-value lab results
CREATE OR REPLACE VIEW critical_lab_results AS
SELECT
    lr.*,
    p.uuid AS patient_uuid,
    p.hospital_id
FROM lab_results lr
JOIN patients p ON lr.patient_id = p.patient_id
WHERE lr.flag = 'Critical'
  AND lr.test_date >= CURRENT_DATE - INTERVAL '7 days'
ORDER BY lr.test_date DESC;

-- Grant permissions
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO
    mediguard_admin;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO
    mediguard_admin;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO mediguard_admin;
```

4.4.3 Apply schema

Apply schema on hospital-a:

```
sudo -u postgres psql -d mediguard_hospital_a -f /home/admin/
    schema.sql
```

Apply schema on hospital-b:

```
sudo -u postgres psql -d mediguard_hospital_b -f /home/admin/
    schema.sql
```

4.5 Synthetic Data Generation

Create file: /home/admin/generate_data.py

```
#!/usr/bin/env python3
"""
MediGuard Synthetic Data Generator
Generates HIPAA-compliant fake patient data
"""

import random
import psycopg2
from faker import Faker
from datetime import datetime, timedelta
import sys

# Initialize Faker
fake = Faker()

# Database connection parameters
DB_CONFIG = {
    'dbname': 'mediguard_hospital_a',  # Change for hospital-b
    'user': 'mediguard_admin',
    'password': 'YourSecurePassword123!',
    'host': 'localhost',
    'port': '5432'
}

# Medical data reference lists
ICD10_CODES = [
    ('J18.9', 'Pneumonia', 'Moderate'),
    ('I10', 'Hypertension', 'Mild'),
    ('E11.9', 'Type 2 Diabetes', 'Moderate'),
    ('J44.9', 'COPD', 'Severe'),
    ('I21.9', 'Myocardial Infarction', 'Critical'),
    ('N18.9', 'Chronic Kidney Disease', 'Severe'),
    ('F32.9', 'Depression', 'Moderate'),
    ('M19.90', 'Osteoarthritis', 'Mild'),
    ('K21.9', 'GERD', 'Mild'),
    ('G43.909', 'Migraine', 'Moderate')
]

DRUGS = [
    ('001', 'Lisinopril', '10mg', '1x daily'),
    ('002', 'Metformin', '500mg', '2x daily'),
    ('003', 'Atorvastatin', '20mg', '1x daily'),
    ('004', 'Omeprazole', '20mg', '1x daily'),
    ('005', 'Albuterol', '90mcg', 'As needed'),
    ('006', 'Warfarin', '5mg', '1x daily'),
    ('007', 'Aspirin', '81mg', '1x daily'),
    ('008', 'Insulin', '10 units', '3x daily')
]
```

```

LAB_TESTS = [
    ('Glucose', 70, 100, 'mg/dL'),
    ('Hemoglobin A1C', 4.0, 5.6, '%'),
    ('Cholesterol', 125, 200, 'mg/dL'),
    ('Blood Pressure Systolic', 90, 120, 'mmHg'),
    ('Creatinine', 0.6, 1.2, 'mg/dL'),
    ('White Blood Cell Count', 4.5, 11.0, '10^9/L')
]
```

```
def generate_patients(conn, num_patients=1000, hospital_id='HOSP-A'):
    """Generate fake patient records (PII will be encrypted later)"""
    cursor = conn.cursor()

    for i in range(num_patients):
        # Generate fake patient data
        gender = random.choice(['M', 'F', 'Other'])
        blood_type = random.choice(['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'
            ])

        # For now, store plaintext (we'll encrypt in Phase 3)
        # Using bytea type with encode() to simulate encrypted storage
        name = fake.name()
        ssn = fake.ssn()
        dob = fake.date_of_birth(minimum_age=18, maximum_age=90)
        address = fake.address().replace('\n', ', ')
        phone = fake.phone_number()

        cursor.execute("""
            INSERT INTO patients (
                name_encrypted, ssn_encrypted, dob_encrypted,
                address_encrypted, phone_encrypted,
                gender, blood_type, hospital_id
            ) VALUES (
                %s::bytea, %s::bytea, %s::bytea,
                %s::bytea, %s::bytea,
                %s, %s, %s
            ) RETURNING patient_id
        """, (
            name.encode(), ssn.encode(), str(dob).encode(),
            address.encode(), phone.encode(),
            gender, blood_type, hospital_id
        ))

        patient_id = cursor.fetchone()[0]

        # Generate 1-5 diagnoses per patient
        num_diagnoses = random.randint(1, 5)
        for _ in range(num_diagnoses):
            icd_code, description, severity = random.choice(ICD10_CODES)
            diag_date = fake.date_between(start_date='-2y', end_date='today')
            doctor_id = random.randint(1, 50)

            cursor.execute("""
                INSERT INTO diagnoses (
                    patient_id, icd10_code, description, severity,
                    diagnosed_date, doctor_id
                ) VALUES (%s, %s, %s, %s, %s, %s)
            """, (patient_id, icd_code, description, severity, diag_date, doctor_id))

        # Generate 0-3 prescriptions per patient
        num_prescriptions = random.randint(0, 3)
        for _ in range(num_prescriptions):
            drug_code, drug_name, dosage, frequency = random.choice(DRUGS)
            rx_date = fake.date_between(start_date='-1y', end_date='today')
            duration = random.randint(7, 90)
            doctor_id = random.randint(1, 50)

            cursor.execute("""
                INSERT INTO prescriptions (
```

```
                    patient_id, drug_code, drug_name, dosage,
                    frequency, duration_days, prescribed_date, doctor_id
                ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, (patient_id, drug_code, drug_name, dosage,
                  frequency, duration, rx_date, doctor_id))
```

```
        # Generate 2-8 lab results per patient
        num_labs = random.randint(2, 8)
        for _ in range(num_labs):
            test_name, min_val, max_val, unit = random.choice(LAB_TESTS)

            # Generate result with 20% chance of abnormal
            if random.random() < 0.2:
                # Abnormal value
                if random.random() < 0.5:
                    result = min_val - random.uniform(10, 30)
                    flag = 'Low'
                else:
                    result = max_val + random.uniform(10, 50)
                    flag = 'High'
            else:
                # Normal value
                result = random.uniform(min_val, max_val)
                flag = 'Normal'

            test_date = fake.date_between(start_date='-1y', end_date='today')

            cursor.execute("""
                INSERT INTO lab_results (
                    patient_id, test_name, result_value, result_unit,
                    normal_range_min, normal_range_max, flag, test_date
                ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, (patient_id, test_name, round(result, 2), unit,
                  min_val, max_val, flag, test_date))

        if (i + 1) % 100 == 0:
            print(f"Generated {i + 1} patients...")
            conn.commit()

    conn.commit()
    cursor.close()
    print(f"Successfully generated {num_patients} patients")

def generate_doctors(conn, num_doctors=50, hospital_id='HOSP-A'):
    """Generate fake doctor records"""
    cursor = conn.cursor()

    specialties = [
        'Cardiology', 'Neurology', 'Oncology', 'Pediatrics',
        'Orthopedics', 'Dermatology', 'Psychiatry', 'General Practice'
    ]

    departments = [
        'Emergency', 'ICU', 'Surgery', 'Outpatient', 'Radiology'
    ]

    for i in range(num_doctors):
        name = fake.name()
        license_num = f"MD{hospital_id[-1]}{random.randint(100000, 999999)}"
        specialty = random.choice(specialties)
        department = random.choice(departments)
        email = fake.email()
        phone = fake.phone_number()

        cursor.execute("""
            INSERT INTO doctors (
                name, license_number, specialty, department,
                hospital_id, email, phone
            ) VALUES (%s, %s, %s, %s, %s, %s, %s)
        """, (name, license_num, specialty, department, hospital_id, email, phone))
```

```
    conn.commit()
    cursor.close()
    print(f"Successfully generated {num_doctors} doctors")
```

```
def main():
    # Check command-line argument for hospital ID
    hospital_id = sys.argv[1] if len(sys.argv) > 1 else 'HOSP-A'

    # Adjust DB config for hospital-b
    if hospital_id == 'HOSP-B':
        DB_CONFIG['dbname'] = 'mediguard_hospital_b'

    print(f"Connecting to {DB_CONFIG['dbname']}...")

    try:
        conn = psycopg2.connect(**DB_CONFIG)
        print("Connected successfully")

        print(f"\nGenerating data for {hospital_id}...")
        generate_doctors(conn, num_doctors=50, hospital_id=hospital_id)
        generate_patients(conn, num_patients=1000, hospital_id=hospital_id)

        # Verify data
        cursor = conn.cursor()
        cursor.execute("SELECT COUNT(*) FROM patients")
        patient_count = cursor.fetchone()[0]
        cursor.execute("SELECT COUNT(*) FROM diagnoses")
        diagnosis_count = cursor.fetchone()[0]
        cursor.execute("SELECT COUNT(*) FROM prescriptions")
        rx_count = cursor.fetchone()[0]
        cursor.execute("SELECT COUNT(*) FROM lab_results")
        lab_count = cursor.fetchone()[0]

        print(f"\n{'='*50}")
        print(f"Data Generation Summary for {hospital_id}")
        print(f"{'='*50}")
        print(f"Patients:      {patient_count:,}")
        print(f"Diagnoses:     {diagnosis_count:,}")
        print(f"Prescriptions: {rx_count:,}")
        print(f"Lab Results:   {lab_count:,}")
        print(f"{'='*50}\n")

        cursor.close()
        conn.close()

    except Exception as e:
        print(f"Error: {e}")
        sys.exit(1)

if __name__ == '__main__':
    main()
```

4.5.1 Install dependencies and run

```
# Install Python libraries
pip3 install psycopg2-binary faker

# Generate data for hospital-a
python3 generate_data.py HOSP-A

# Generate data for hospital-b (run on hospital-b VM)
python3 generate_data.py HOSP-B
```

4.6 Configure Streaming Replication

4.6.1 On hospital-a (Primary Server)

```
# Edit postgresql.conf
sudo vim /etc/postgresql/16/main/postgresql.conf
```

Add/modify:

```
wal_level = replica
max_wal_senders = 3
max_replication_slots = 3
hot_standby = on
```

4.6.2 Create replication user

```
sudo -u postgres psql
```

```
CREATE ROLE replicator WITH REPLICATION PASSWORD '
    ReplicatorPass123!' LOGIN;
\q
```

4.6.3 Edit pg_hba.conf

```
sudo vim /etc/postgresql/16/main/pg_hba.conf
```

Add:

```
host replication replicator 10.0.0.1/32 scram-sha-256
```

4.6.4 Restart PostgreSQL

```
sudo systemctl restart postgresql
```

4.6.5 Create standby replica (on same VM, different port)

Note

This is a simplified setup. In production, the replica would be on a separate server.

```
# Stop replica instance (if running)
sudo systemctl stop postgresql@16-replica

# Create replica data directory
sudo mkdir -p /var/lib/postgresql/16/replica
sudo chown postgres:postgres /var/lib/postgresql/16/replica
```

```
7
8  # Create base backup
9  sudo -u postgres pg_basebackup -h localhost -D /var/lib/
       postgresql/16/replica \
10   -U replicator -v -P --wal-method=stream
11
12 # Create standby signal file
13 sudo -u postgres touch /var/lib/postgresql/16/replica/standby.
       signal
14
15 # Create replica config
16 sudo cp /etc/postgresql/16/main/postgresql.conf \
17   /etc/postgresql/16/replica/postgresql.conf
18
19 # Edit replica config to use different port
20 sudo vim /etc/postgresql/16/replica/postgresql.conf
```

Change:

```
1  port = 5433
2  primary_conninfo = 'host=localhost␣port=5432␣user=replicator␣
       password=ReplicatorPass123!'
```

4.6.6 Start replica

```
1  sudo -u postgres pg_ctl -D /var/lib/postgresql/16/replica start
```

4.6.7 Verify replication

```
1  sudo -u postgres psql -c "SELECT * FROM pg_stat_replication;"
```

Expected output: Shows active replication connection

Note

Repeat similar setup on `hospital-b`

4.7 Test Database Functionality

4.7.1 Test stored procedures

```
1  sudo -u postgres psql -d mediguard_hospital_a
```

```
1  -- Test 1: Flag high-risk patients
2  SELECT * FROM flag_high_risk_patients() LIMIT 10;
3
4  -- Test 2: Check drug interactions
```

```
SELECT * FROM check_drug_interactions(1);

-- Test 3: Outbreak report
SELECT * FROM generate_outbreak_report('HOSP-A', 12);

-- Test 4: Audit log
SELECT secure_audit_insert('admin', 'SELECT * FROM patients
   LIMIT 10', '10.0.0.3');
SELECT * FROM audit_log ORDER BY timestamp DESC LIMIT 5;

\q
```

4.8 Phase 2 Verification Checklist

- ☐ PostgreSQL 16 installed on both hospital VMs
- ☐ Databases created (`mediguard_hospital_a`, `mediguard_hospital_b`)
- ☐ Schema applied successfully
- ☐ 1000+ synthetic patients generated on each hospital
- ☐ Stored procedures execute without errors
- ☐ Streaming replication shows active connection
- ☐ Audit log is append-only (UPDATE/DELETE blocked)
- ☐ Database accessible from ml-server via VPN

4.9 Troubleshooting

Problem	**Solution**
Connection refused	Check `listen_addresses` in `postgresql.conf`
Authentication failed	Verify password in `pg_hba.conf` matches user password
Replication not working	Check firewall allows port 5432, verify `primary_conninfo`
Stored procedure fails	Check function syntax with `\df function_name`

Table 4.1: Common Database Issues and Solutions